PRAKTIKUM 8
POLYMORPHISM

NAMA: Ihsanta Zaki Sanjaya

KELAS: 1 D4 Teknik Komputer B

NRP: 3223600058

DOSEN: Dr Bima Sena Bayu Dewantara S.ST, M.T

MATA KULIAH: Praktikum Pemograman Dasar 2

TANGGAL: Rabu, 8 Mei 2024

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2024

Praktikum 8
Polymorphism

Untuk memberikan pemahaman kepada anda tentang konsep dasar polymorphism dan cara menggunakan dalam program C++, maka cobalah beberapa modul percobaan dibawah ini.

1. Percobaan 1 – Polymorphism

```
#include <iostream>

using namespace std;

class Shape
{
	protected:
	int width, height;

	public:
	Shape( int a=0, int b=0)
	{
		width = a;
		height = b;
	}

	int area()
	{
		cout << "Parent class area :" <<endl;
		return 0;
	}
};

class Rectangle: public Shape
{
	public:
	Rectangle( int a=0, int b=0): Shape(a, b) { }

	int area ()
	{
		cout << "Rectangle class area :" <<endl;
		return (width * height);
```

```
        }
};

class Triangle: public Shape
{
        public:
        Triangle( int a=0, int b=0): Shape(a, b) { }

        int area ()
        {
                 cout << "Triangle class area :" <<endl;
                return (width * height / 2);
        }
};

// Main function for the program
int main( )
{
        Shape *shape;
        Rectangle rec(10, 7);
        Triangle tri(10, 5);

        // store the address of Rectangle
        shape = &rec;

        // call rectangle area.
        shape->area();

        // store the address of Triangle
        shape = &tri;

        // call triangle area.
        shape->area();

        return 0;
}
```

Hasil percobaan:

Praktikum > P9 1.2 waduh.cpp > ...

```cpp
#include <iostream>
using namespace std;
class Shape {
    protected:
    int width,height;
    public:
    Shape(int a=0,int b=0) {
        width=a; height=b;
    }
    int area() {
        cout<<"Parent class area :"<< endl;
        return 0;
    }
};
class Rectangle:public Shape {
public:
    Rectangle(int a=0,int b=0):Shape(a, b) {}
    int area() {
        cout<<"Rectangle class area :"<< endl;
        return(width*height);
    }
};
class Triangle:public Shape {
public:
    Triangle(int a=0,int b=0):Shape(a, b) {}
    int area() {
        cout<<"Triangle class area :"<< endl;
        return(width*height/2);
    }
};
int main() {
    Shape *shape;
    Rectangle rec(10,7);
    Triangle tri(10,5);
    shape=&rec;
    shape->area();
    shape=&tri;
    shape->area();
    return 0;
}
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\
Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1
.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher
.exe' '--stdin=Microsoft-MIEngine-In-4oxh55if.sff' '--
stdout=Microsoft-MIEngine-Out-4az5plyf.01v' '--stderr=
Microsoft-MIEngine-Error-grylerig.zex' '--pid=Microsof
t-MIEngine-Pid-rhyahtza.3wl' '--dbgExe=C:\msys64\ucrt6
4\bin\gdb.exe' '--interpreter=mi'
Parent class area :
Parent class area :
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Ouput program:

```
Parent class area :
Parent class area :
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Pada percobaan ini metode area() bekerja dengan cara yang berbeda tergantung pada apakah itu dipanggil pada objek Rectangle atau Triangle. Meskipun Rectangle dan Triangle memiliki metode area() mereka sendiri, ketika metode tersebut dipanggil melalui pointer Shape, metode area() dari kelas Shape yang dipanggil, bukan dari Rectangle atau Triangle. Ini karena metode area() dalam kelas Shape tidak dideklarasikan sebagai virtual. Percobaan ini berfungsi untuk memahami konsep dasar dari pewarisan dan polimorfisme dalam pemrograman berorientasi objek.

2. Percobaan 2 – Virtual Function

Modifikasilah sedikit pada percobaan 1 di atas dengan baris perintah seperti di bawah ini.

```
class Shape
{
        protected:
        int width, height;

        public:
        Shape( int a=0, int b=0)
        {
                width = a;
                height = b;
        }

        virtual int area()
        {
                cout << "Parent class area :" <<endl;
                return 0;
        }
};
```

Hasil percobaan:

```
Praktikum > P9 coba2.cpp > Shape > area()
1  #include <iostream>
2  using namespace std;
3  class Shape {
4      protected:
5      int width,height;
6      public:
7      Shape(int a=0,int b=0) {
8          width=a; height=b;
9      }
10     virtual int area() {
11         cout<<"Parent class area :"<< endl;
12         return 0;
13     }
14 };
15 class Rectangle:public Shape {
16 public:
17     Rectangle(int a=0,int b=0):Shape(a, b) {}
18     int area() {
19         cout<<"Rectangle class area :"<< endl;
20         return(width*height);
21     }
22 };
23 class Triangle:public Shape {
24 public:
25     Triangle(int a=0,int b=0):Shape(a, b) {}
26     int area() {
27         cout<<"Triangle class area :"<< endl;
28         return(width*height/2);
29     }
30 };
31 int main() {
32     Shape *shape;
33     Rectangle rec(10,7);
34     Triangle tri(10,5);
35     shape=&rec;
36     shape->area();
37     shape=&tri;
38     shape->area();
39     return 0;
40 }
41
```

```
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>  & 'c:\
Users\MyLaptop\.vscode\extensions\ms-vscode.cpptools-1
.19.9-win32-x64\debugAdapters\bin\WindowsDebugLauncher
.exe' '--stdin=Microsoft-MIEngine-In-bubbvrak.cby' '--
stdout=Microsoft-MIEngine-Out-fnyez1rj.pkl' '--stderr=
Microsoft-MIEngine-Error-ia15fxgk.1wv' '--pid=Microsof
t-MIEngine-Pid-a2lngito.pdh' '--dbgExe=C:\msys64\ucrt6
4\bin\gdb.exe' '--interpreter=mi'
Rectangle class area :
Triangle class area :
PS D:\1. KULIAH PENS\Semester II\3. Progdas 2>
```

Output program:

```
Rectangle class area :
Triangle class area :
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Pada percobaan ini metode area() bekerja dengan cara yang berbeda tergantung pada apakah itu dipanggil pada objek Rectangle atau Triangle. Karena metode area() dalam kelas Shape dideklarasikan sebagai virtual, maka metode area() yang sesuai dari kelas anak yang dipanggil (Rectangle atau Triangle), bukan dari kelas Shape.

3. Percobaan 3 – Pure Virtual Function

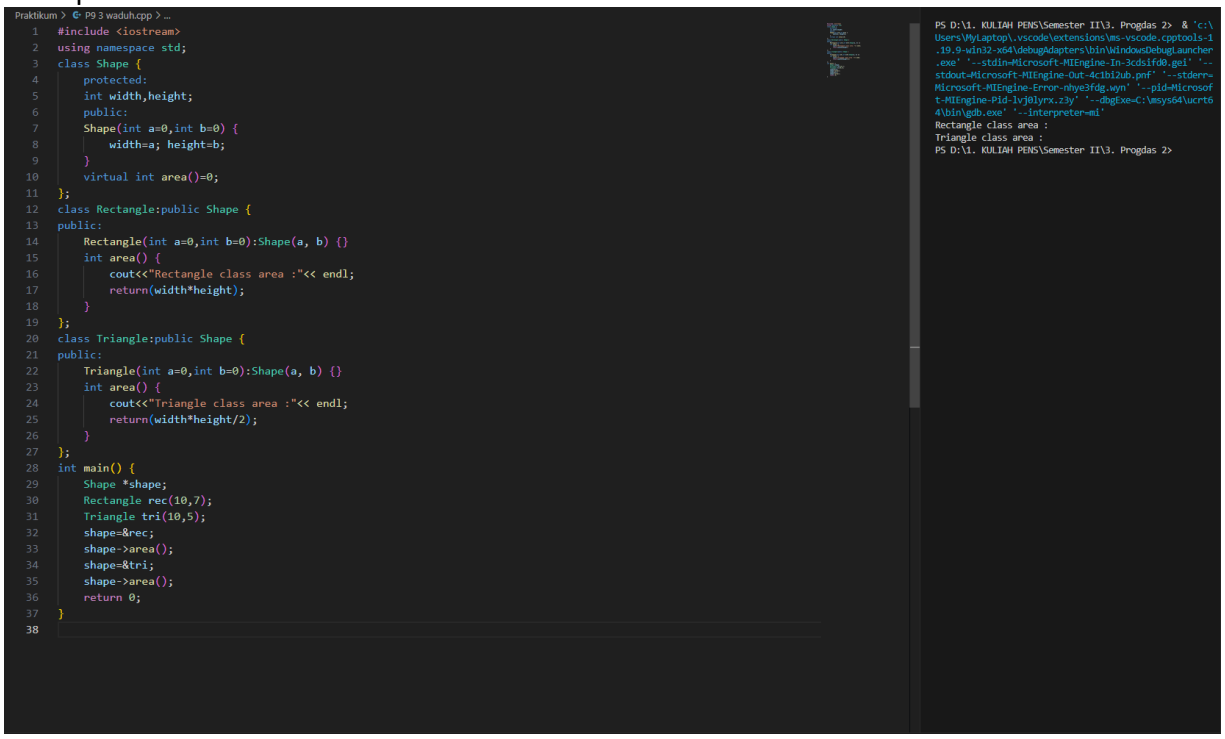
Sekali lagi, modifikasilah sedikit pada percobaan 1 di atas dengan baris perintah seperti di bawah ini.

```
class Shape
{
        protected:
        int width, height;

        public:
        Shape( int a=0, int b=0)
        {
                width = a;
                height = b;
        }

        // pure virtual function
        virtual int area() = 0;
};
```

Hasil percobaan:



Output ptogram:

```
Rectangle class area :
Triangle class area :
```

Dari percobaan diatas, apakah yang dapat anda simpulkan? Jelaskan!

Kelanjutan dari percobaan sebelumnya, pada percobaan ini metode area() dalam kelas Shape dideklarasikan sebagai fungsi virtual murni. Fungsi ini harus diimplementasikan oleh semua kelas turunan non-abstract. Deklarasi fungsi virtual murni mengubah kelas Shape menjadi kelas abstrak. Kelas ini hanya dapat digunakan sebagai kelas dasar untuk kelas lain. Sehingga, metode area() harus diimplementasikan oleh semua kelas yang diturunkan dari

Shape, yaitu Rectangle dan Triangle. Jika ada kelas turunan yang tidak mengimplementasikan metode area(), maka kelas tersebut juga akan menjadi kelas abstrak. Jadi, perbedaan utama dengan kode sebelumnya adalah bahwa dalam kode ini, kita tidak dapat membuat objek dari kelas Shape karena sekarang adalah kelas abstrak. Yang hanya dapat dibuat adalah objek dari kelas Rectangle dan Triangle, dan metode area() yang sesuai akan dipanggil tergantung pada jenis objek yang merujuk.